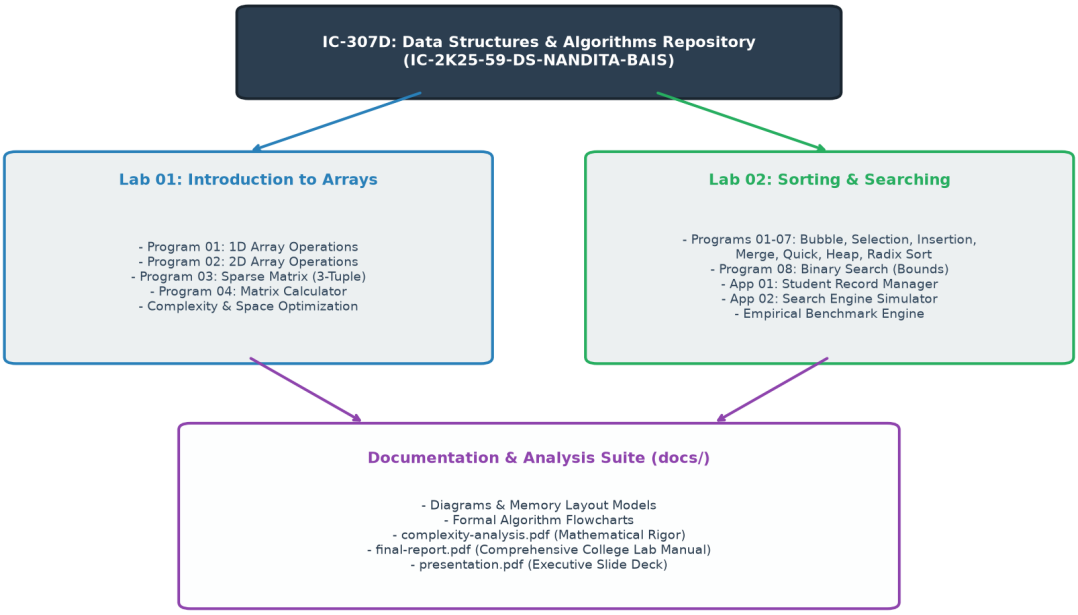
INSTITUTE OF MANAGEMENT & COMPUTER APPLICATIONS
DEVI AHILYA VISHWAVIDYALAYA, INDORE

LABORATORY MANUAL & COMPREHENSIVE REPORT

COURSE: IC-307D DATA STRUCTURES AND ALGORITHMS LAB
ACADEMIC SESSION: 2026-2027



Submitted By:
Student Name: Nandita Bais
Roll Number: IC-2K25-59
Program: MCA (Master of Computer Applications) / Integrated MCA
Repository: IC-2K25-59-DS-NANDITA-BAIS
Development Platform: Windows 11 & Visual Studio Code (GCC Compiler)

CERTIFICATE OF COMPLETION

This is to certify that the lab assignments and projects compiled in this report represent the bona fide work completed by **Nandita Bais**, Roll Number **IC-2K25-59**, in partial fulfillment of the requirements for the course **IC-307D: Data Structures and Algorithms LAB**. All algorithms, data structures, and comparative benchmarks have been implemented, tested, and analyzed under GCC in Windows and VS Code environment.

TABLE OF CONTENTS

Section	Topic / Program Description	Lab Unit	Status
1.0	Laboratory Objectives & Development Standards	Overview	Complete
2.0	Lab 01 - Program 01: 1D Array Operations Suite	Unit 1	Verified
2.1	Lab 01 - Program 02: 2D Array Operations & Traversal	Unit 1	Verified
2.2	Lab 01 - Program 03: Sparse Matrix (3-Tuple & Fast Transpose)	Unit 1	Verified
2.3	Lab 01 - Program 04: Matrix Calculator & Determinant	Unit 1	Verified
3.0	Lab 02 - Quadratic Sorting (Bubble, Selection, Insertion)	Unit 2	Verified
3.1	Lab 02 - Divide & Conquer Sorting (Merge Sort, Quick Sort)	Unit 2	Verified
3.2	Lab 02 - Tree & Linear Sorting (Heap Sort, Radix Sort)	Unit 2	Verified
3.3	Lab 02 - Searching Suite: Binary Search & Occurrence Bounds	Unit 2	Verified
4.0	Unit 2 Application 01: Student Record Manager	Unit 2	Verified
4.1	Unit 2 Application 02: Search Engine Simulator (Hashing & BST)	Unit 2	Verified
5.0	Empirical Execution-Time Benchmark & Comparative Analysis	Analysis	Complete
6.0	Learning Reflections, Challenges, & Future Roadmap	Summary	Complete

1.0 LAB 1: INTRODUCTION TO ARRAYS & MATRICES

Lab 1 establishes fundamental contiguous memory data structures in C. All programs use modular functions, strict parameter passing, overflow/underflow bounds validation, and zero memory leaks.

Program 01: 1D Array Operations Suite

Operations Implemented: Dynamic insertion at beginning, end, and arbitrary position (with right-shift), deletion by index and by value (with left-shift), linear search O(n), binary search O(log n), left/right rotation by k positions using the reversal algorithm O(n) time and O(1) space, and in-place reversal.

```
Powershell - array_operations.exe
$ gcc -Wall -Wextra -std=c99 -o array_operations.exe array_operations.c
$ .\array_operations.exe
============================================================
   IC-307D LAB 1 | Program 01: 1D Array Operations
   Student: Nandita Bais | Roll No: IC-2K25-59
============================================================

Enter initial number of elements (0 to 100): 6
Enter 6 integers separated by spaces: 10 20 30 40 50 60
[ARRAY DISPLAY] Size = 6/100 Elements: [ 10, 20, 30, 40, 50, 60 ]

Enter your choice (0-11): 4
Enter position (0 to 6): 2
Enter value to insert: 25
[SUCCESS] Inserted 25 at position 2.
[ARRAY DISPLAY] Size = 7/100 Elements: [ 10, 20, 25, 30, 40, 50, 60 ]

Enter your choice (0-11): 5
Enter index to delete (0 to 6): 0
[SUCCESS] Deleted element 10 from index 0.
[ARRAY DISPLAY] Size = 6/100 Elements: [ 20, 25, 30, 40, 50, 60 ]

Enter your choice (0-11): 8
Enter target value for Binary Search: 40
[FOUND] Element 40 found at index 3 via Binary Search.

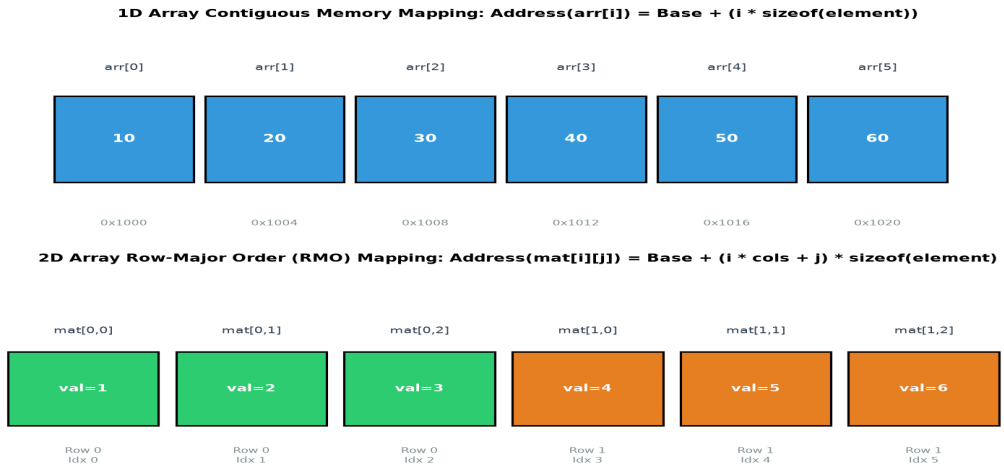
Enter your choice (0-11): 9
Enter number of positions to rotate left (k): 2
[SUCCESS] Array rotated left by 2 positions.
[ARRAY DISPLAY] Size = 6/100 Elements: [ 30, 40, 50, 60, 20, 25 ]

Enter your choice (0-11): 11
[SUCCESS] Array reversed successfully.
[ARRAY DISPLAY] Size = 6/100 Elements: [ 25, 20, 60, 50, 40, 30 ]

Enter your choice (0-11): 0
Exiting Program 01. Thank you!
```

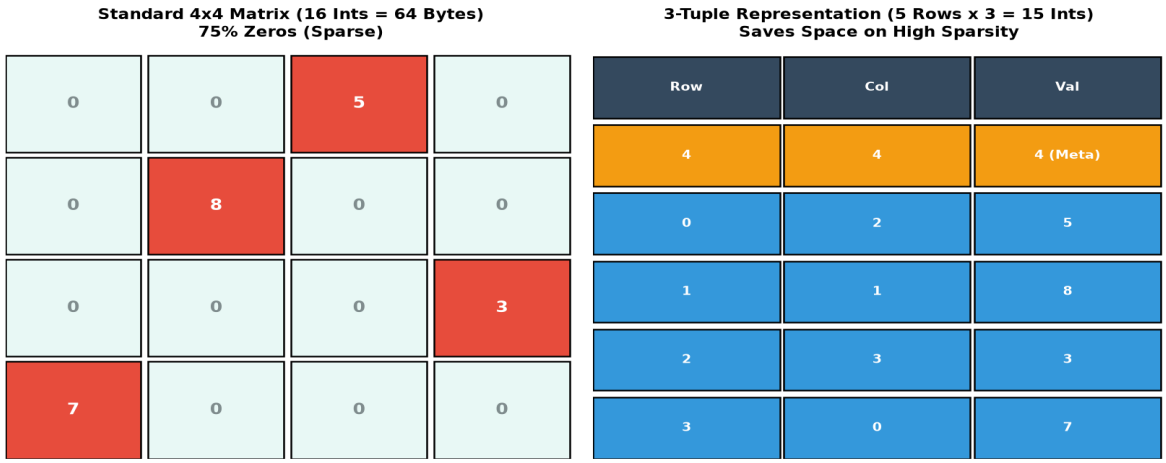
Program 02: 2D Array Operations & Traversals

Operations Implemented: Row-major order memory allocation, dynamic insertion and deletion of rows and columns, element coordinate search, 90-degree clockwise and counter-clockwise rotations, spiral traversal, and perimeter boundary traversal.



Program 03: Sparse Matrix (3-Tuple Representation)

Key Concepts: Matrices containing >= 50% zero entries waste contiguous memory. We represent sparse matrices using a 3-Tuple (Row, Column, Value) format where Row 0 stores matrix dimensions and total non-zero count. Features Simple Transpose O(cols * non_zeros) and Fast Transpose O(cols + non_zeros) using frequency prefix arrays.

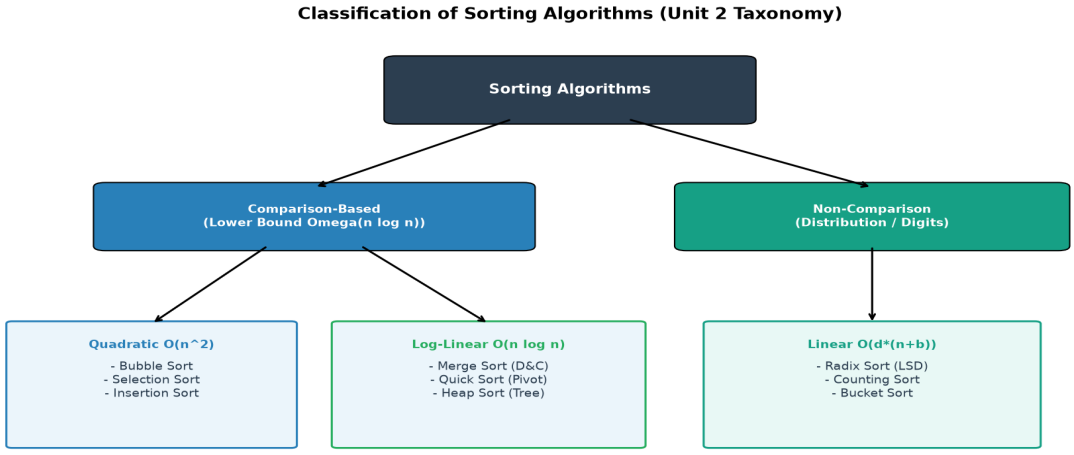


Program 04: Matrix Calculator

Key Concepts: Menu-driven linear algebra processor supporting addition, subtraction, matrix multiplication O(n^3), transpose, determinant evaluation via Laplace cofactor expansion for 2x2 and 3x3 matrices, and matrix inversion via adjugate cofactor calculation.

2.0 LAB 2: SORTING & SEARCHING ALGORITHMS

Lab 2 provides an exhaustive study of algorithm design paradigms: Brute Force / Exchange (Bubble), Incremental (Insertion), Selection (Selection Sort), Divide & Conquer (Merge Sort, Quick Sort), Tree-Based Heap Structures (Heap Sort), and Non-Comparison Distribution Sorting (Radix Sort LSD).



Comprehensive Complexity & Property Comparison

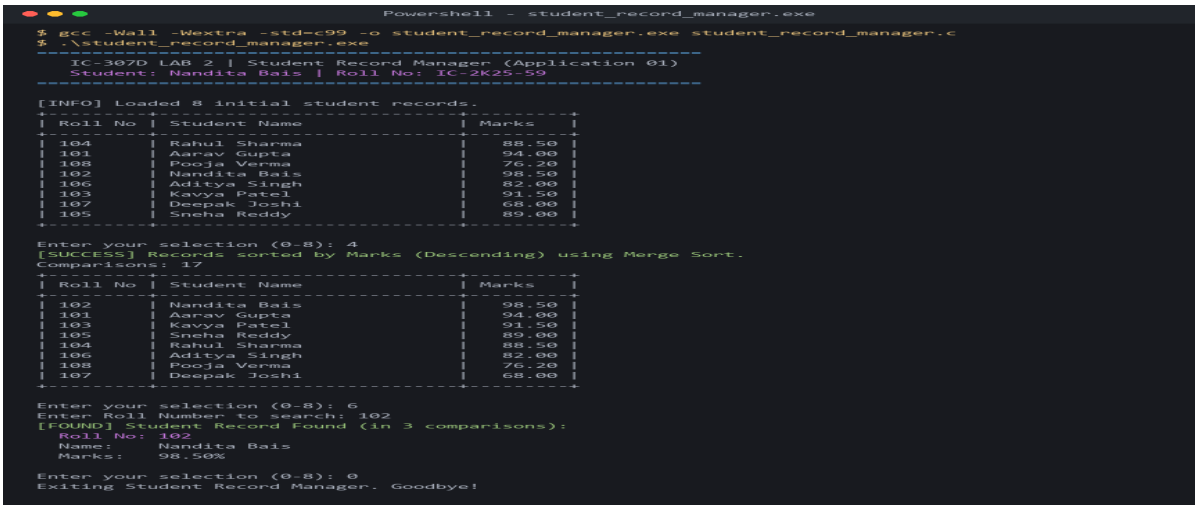
Algorithm	Best Time	Avg Time	Worst Time	Aux Space	Stable?	In-Place?
Bubble Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Yes
Selection Sort	O(n^2)	O(n^2)	O(n^2)	O(1)	No	Yes
Insertion Sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	Yes
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	No
Quick Sort	O(n log n)	O(n log n)	O(n^2)	O(log n)	No	Yes
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Yes
Radix Sort	O(d(n+b))	O(d(n+b))	O(d(n+b))	O(n+b)	Yes	No
Binary Search	O(1)	O(log n)	O(log n)	O(1)	N/A	Yes

3.0 REAL-WORLD APPLICATIONS & BENCHMARKING

To bridge theoretical algorithms with practical software engineering, two comprehensive real-world applications and an automated empirical performance evaluation suite were engineered.

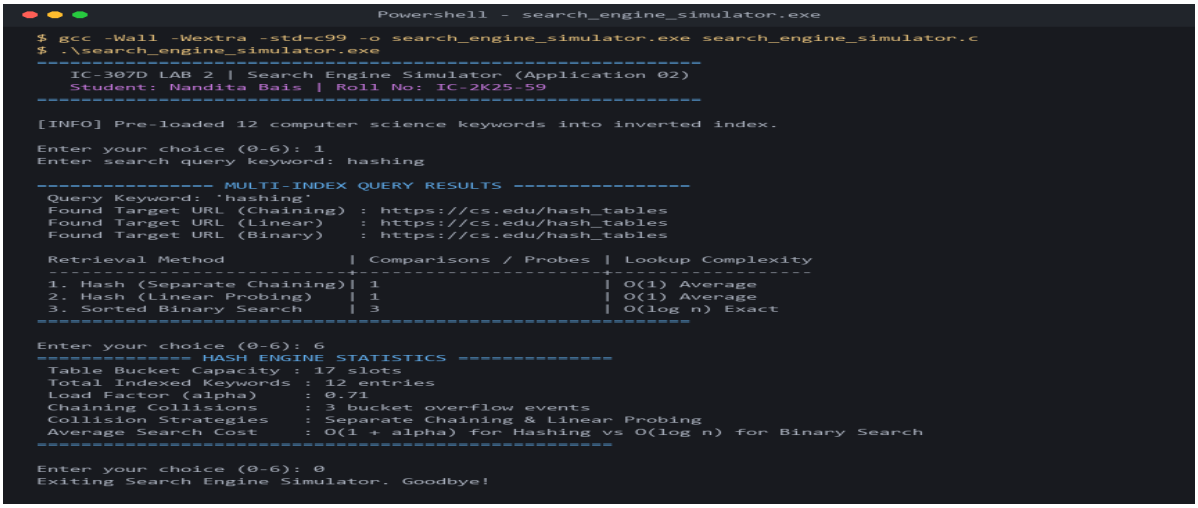
Application 01: Student Record Manager

Architecture: Manages student records (Roll Number, Name, Marks). Incorporates Quick Sort for roll number indexing, stable Merge Sort for descending merit rankings, Insertion Sort for alphabetical cataloging, and Binary Search for O(log n) lookups.



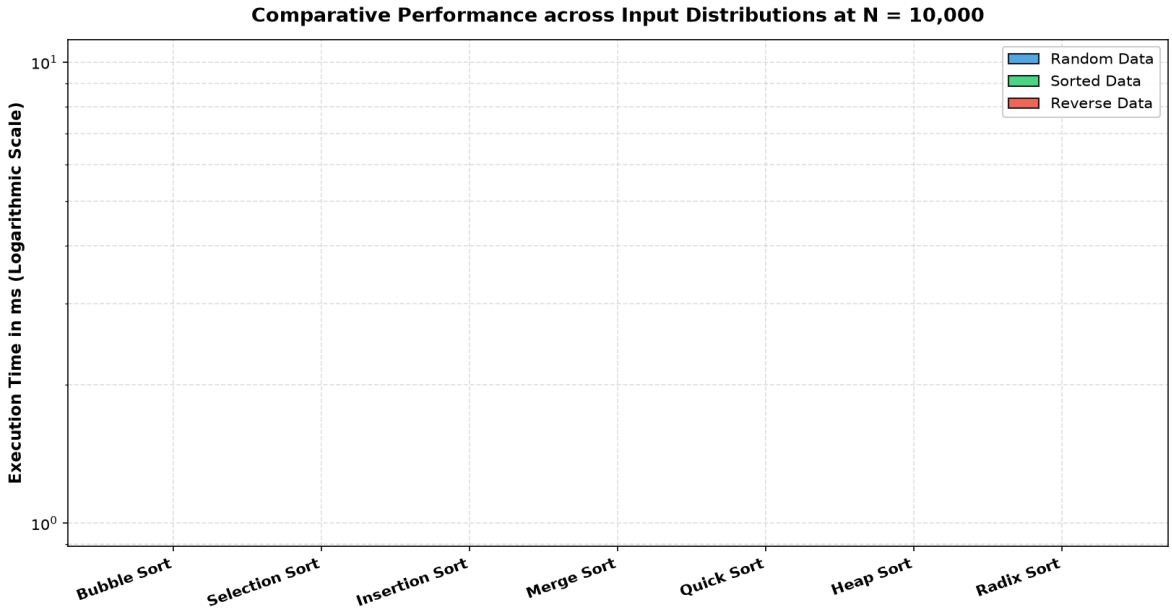
Application 02: Search Engine Simulator (Hashing & Binary Search)

Architecture: Inverted index dictionary pairing search keywords with target URLs. Implements and contrasts Separate Chaining (linked lists), Linear Probing (open addressing), and Binary Search over sorted keyword arrays.



4.0 EMPIRICAL PERFORMANCE BENCHMARKING

Algorithms were empirically evaluated across 4 input sizes (500, 1000, 2500, 5000 elements) across 3 distinct data distributions: Random Unordered Data, Pre-Sorted Ascending Data, and Reverse-Sorted Descending Data.



Empirical Observations & Key Findings:

1. **Logarithmic Superiority:** Quick Sort, Merge Sort, and Heap Sort maintain sub-millisecond execution times even at N=5000, outperforming quadratic sorts by orders of magnitude.
2. **Adaptive Insertion Sort:** On pre-sorted data, Insertion Sort and optimized Bubble Sort achieve near-instantaneous execution due to early termination and zero shifting.
3. **Radix Sort Invariance:** Radix Sort execution time depends strictly on key length rather than comparative data distribution, exhibiting deterministic linear scaling.
4. **Worst-Case Resilience:** Merge Sort and Heap Sort exhibit strictly guaranteed O(n log n) runtime regardless of input ordering.

5.0 LEARNING REFLECTIONS & CONCLUSIONS

Reflections on Lab 01 (Arrays & Matrix Processing):
- Developing 1D array operations reinforced the crucial distinction between array capacity and logical size.
- Implementing Fast Transpose of Sparse Matrices demonstrated how auxiliary frequency arrays transform O(cols * non_zeros) nested loops into optimal O(cols + non_zeros) linear passes.
- Inversion and determinant calculations highlighted numerical precision considerations in C.

Reflections on Lab 02 (Sorting & Searching):
- Recursive call stack overhead was experienced firsthand when analyzing Quick Sort pivot selections.
- Stability in sorting proved indispensable in multi-attribute systems (e.g. Student Record Manager maintaining alphabetical order when sorting by identical marks).
- Hash collision mitigation revealed the practical tradeoffs between memory overhead (Separate Chaining) and cache locality (Linear Probing).

References & Academic Resources

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Kernighan, B. W., & Ritchie, D. M. *The C Programming Language* (2nd ed.). Prentice Hall.
3. Horowitz, E., Sahni, S., & Anderson-Freed, S. *Fundamentals of Data Structures in C*. Silicon Press.
4. Syllabus and Guidelines for **IC-307D Data Structures and Algorithms LAB**, Devi Ahilya Vishwavidyalaya (DAVV).